



Fleet Management Dashboard: Hiring Challenge

SDE-1 role, Peppermint Robotics

This is a hiring challenge for the **SDE-1** role at **Peppermint Robotics**. We are hiring full stack engineers, so solve only one of the two assignments below, whichever you would rather work in, and answer the system design questions at the end regardless of which one you pick.

How this works

- **Time.** We expect roughly 6 to 10 focused hours; the backend assignment usually runs longer than the frontend one. Timebox yourself and stop there. We would rather see part of this done well than all of it done in a rush: an honest, well reasoned gap scores better than a complete submission that cuts corners quietly. Write down what you cut and why.
- **Deadline.** Send your submission within 4 days of receiving this package. The timebox above fits comfortably in that window, and part of what we are gauging is how quickly you can pick up an unfamiliar problem and ship something honest. If something genuinely blocks you, tell us before the deadline, not after.
- **How to submit.** Email kautilya.boga@peppermintrobotics.com with a link to a git repository, or an archive of your source.
- **What happens next.** We review every submission and acknowledge it. If you are shortlisted, we set aside up to half an hour for a walkthrough call: you demo your work live, we ask about specific decisions you made, and we will ask you to make a small change to your code during the call.
- **AI tooling.** Use any AI tooling you are comfortable with, freely but carefully. Note in your README which parts you delegated to AI. AI use is not penalized; inability to explain any part of your own submission is.
- **Managed services.** Modern managed services (Supabase, Firebase, hosted message brokers, Railway, anything) are welcome wherever they fit, as long as the required deliverable still works as specified below and you can explain what the service is doing for you, and what you would do if it disappeared.
- **Your work stays yours.** The data in this package is synthetic. Your code remains yours and you may keep it public in your portfolio. We use submissions only for evaluation.

The data

Both assignments use the same three files, since they describe the same fleet and site:

- `layout.png` is the site, as a plain image. The origin, $(0, 0)$, is the top left corner; x increases to the right and y increases downward, matching standard image coordinates. One pixel equals one unit of distance, so there is no scale conversion involved.
- `robots.json` is a fixed roster of eight robots, each with a `robot_id`, a `robot_type`, and a starting position in the same coordinate system as the image.
- `events.jsonl` is a recorded log covering fifteen minutes, one JSON object per line: each robot reports its position, battery percentage, and status roughly every five seconds. A typical line looks like this:

```
{"t": 55, "robot_id": "r6", "x": 602.7, "y": 344.8, "status":  
"on_mission", "battery": 43.7}
```

`t` is seconds from the start of the window (0 to 900). `battery` is a percentage. `status` is one of `idle`, `active`, `on_mission`, `charging`, `blocked`, `error`, `maintenance`, or `offline`.

A couple of lines also carry an extra `task_event` key (`task_started` or `task_completed`). They are rare in this window and may be unpaired. Nothing is graded on them; surface them or ignore them, your call.

We deliberately do not define which statuses count as working or as needing attention, or what `active` means versus `on_mission`. Make a sensible call and be ready to defend it.

This log comes from a small scripted simulation on our end, so there is nothing hidden in it you need to reverse engineer.

Assignment 1: Frontend

We are asking you to build the frontend for a fleet management dashboard. This service will be used by an operator who is responsible for a fleet of robots that move around a site. Each robot reports a battery percentage, a position, and a status describing what it is currently doing. You may use any language or framework.

The live feed

Alongside replaying `events.jsonl`, simulate a live feed of new events for the same robots. Keep emitting plausible new events (continuous movement, gradual battery change, sensible status transitions) at a rate you choose; replaying the recorded file again does not count as a live feed. Whether you generate it from a small server process or purely within the frontend is up to you. Both the replay and the live feed need to be reachable in your deployed submission. Tell us briefly how you

approached it and roughly what rate you used.

What the dashboard needs to do

An operator using this dashboard, watching either the replayed log or your live feed, needs to be able to:

1. **See the site and the robots on it, as they move**, legibly, with all eight visible at once. Replay faster than real time or provide a speed control; we will not watch fifteen minutes of playback in real time.
2. **See how things are trending** over the observed window. An operator here is typically less interested in any one robot's history and more in fleet level questions, for instance the fraction of the fleet active, shown over time. Show at least one such trend. A single current value readout does not count as a trend.
3. **Find a specific robot, or the ones that need attention**, and see enough about it to decide what to do next.

Written answers

Send these back in a short `ANSWERS.md`, a paragraph or two each. Answer with reference to your own code; name the files or functions where each decision lives.

1. What holds the fleet's state as data arrives, and why that shape, given that both the replay and your live feed need to drive the same views?
2. Name one real tradeoff you made while building this, and argue for the decision. What did it cost you?
3. What did you leave out, and what would you build next given more time?

Submitting your work

Send us a git repository or archive containing your source, a `README.md` covering how to install and run it, your AI delegation notes, and what you would do next. Include a couple of tests for the part you found trickiest. **You need to include a working link to your deployed submission.** The frontend is easy to host for free (GitHub Pages, Netlify, Vercel, or similar), and we want to be able to open it without setting anything up. If your live feed comes from a server process, either deploy that too (Render, Fly.io, or similar) or make the deployed build fall back to generating the feed in the browser. Verify your deployed link in a private browser window before submitting.

Checklist: source repo or archive • `README.md` (run steps + AI delegation notes) • `ANSWERS.md` • `SYSTEM_DESIGN.md` • working deployed link • tests for the trickiest part

Assignment 2: Backend

We are asking you to build the backend for a fleet management dashboard, for the same operator and the same fleet. In this assignment the data does not arrive as a file: each robot publishes it directly, live, and it is on you to get it from the robots to whoever needs to consume it. You may use any language or framework.

Use `events.jsonl` to drive your mocked robots. Each simulated robot should publish its own recorded events, in order, standing in for what that robot would report live. You can publish at the recorded pace, faster, or on your own timer; the point is that your publishers are sourced from this log rather than fully invented, so the fleet you are serving is the same one Assignment 1 is drawing.

What you are building

1. **Mock the robots.** Run a small simulated fleet with one publisher per robot in `robots.json`, each publishing its `robot_id`, position, battery percentage, and status by replaying its events from `events.jsonl`. Eight compose services is fine; one container that spawns eight processes is fine; eight coroutines inside a single process is not what we mean. How the robots get that data to your backend is entirely your call: MQTT, a message queue, plain sockets, HTTP callbacks, anything that fits a producer and consumer split. Tell us what you picked and why.
2. **Run a backend service that consumes that feed.** It should ingest every robot's updates through whatever mechanism you designed above, and maintain the fleet's current state.
3. **Expose that state two ways**, so a consumer can opt for either: a **WebSocket** stream that pushes updates in real time, and a **REST endpoint** that returns current state on request, for a consumer that prefers to poll. Both need to reflect the same underlying state; a client using one should not see something inconsistent with a client using the other.

Package it with Docker Compose. `docker compose up` should bring up everything this needs: your backend service, your mocked robot fleet, and any broker or message system your architecture relies on, as separate services, with no manual setup beyond having Docker installed. One of those services should run a script that starts the robot simulation; we should not need to start it by hand in a separate terminal. If you lean on a hosted managed service for part of this, the core system must still come up with this one command.

Real deployments have flaky networks: a robot's connection can drop and reconnect, and so can a WebSocket client's. Your design should account for that rather than assume a clean, always on connection. Deployment beyond your own machine is not required; Docker Compose running locally is the whole submission.

Our evaluation environment: we run submissions on x86_64 Linux with a recent Docker. If you build on ARM (Apple Silicon), pin your image platforms so it runs for us. If your stack does not boot on our machine, we will reply with the error and give you one chance to fix it.

Optional stretch goal

If you finish inside the timebox and want headroom: persist the fleet history and expose it as `GET /robots/history/{robot_id}` with a time range. Any store is fine (SQLite, Postgres, Supabase, anything); one sentence on why you chose it. Optional means optional; do not blow the timebox on it.

Written answers

Send these back in a short `ANSWERS.md`, a paragraph or two each. Answer with reference to your own code; name the files or functions where each decision lives.

1. What holds the fleet's current state in your backend, and why that shape, given it has to serve both the WebSocket stream and the polling endpoint consistently?
2. Name one real tradeoff you made: the mechanism you chose for robots to reach your backend, its delivery guarantees, and how you reconcile that mechanism's semantics with your WebSocket fanout. Argue for the decision, including its cost.
3. What did you leave out, and what would you build next given more time?

Submitting your work

Send us a git repository or archive containing your source, a `docker-compose.yml` that brings the whole thing up with one command, and a `README.md` covering your design decisions, your AI delegation notes, and what you would do next. Include a couple of tests for the part you found trickiest.

Checklist: source repo or archive • `docker-compose.yml` • `README.md` (design decisions + AI delegation notes) • `ANSWERS.md` • `SYSTEM_DESIGN.md` • tests for the trickiest part

System design questions

Answer these in writing, a paragraph or two each, in a file called `SYSTEM_DESIGN.md`, alongside the rest of your submission. They build on the system you have just built, whichever assignment you took and whatever stack you chose. Answer with reference to that actual system: name the file, function, or service where each answer would land. Generic essays that could be written without having built anything score poorly.

1. What happens if we ask you to add a new feature to this later? Does your current design accommodate that, or does it need a rework? Walk through a specific feature and where it would plug in.
2. What happens if the number of robots grows a lot, say from eight to five hundred? What is the first thing that breaks, and why that specifically?
3. What happens if bandwidth is limited and robots and the backend can only exchange a small amount of data per second? What would you change about what you send, how often, or how

much detail it carries?

4. What happens if a robot goes down mid task and stops responding? What should the rest of the system do about it, and how would it even find out?
 5. What happens if the connection between a robot and the backend is slow or unreliable, and updates arrive late, out of order, or not at all for a while? What does the rest of the system see during that time, and how does it recover once the connection is healthy again?
-

That is everything. If anything in this document is unclear, write to kautilya.boga@peppermintrobotics.com and ask; a good clarifying question costs you nothing. We look forward to seeing what you build.